



Annee universitaire 2025–2026

Travaux Pratiques

TP 4

Usine Logicielle DevOps Complete

Pipeline CI/CD de A a Z

CI / CD

INFRASTRUCTURE AS CODE

KUBERNETES

OBSERVABILITY

Etudiant

Kacem Mathlouthi

Etudiant

Oussema Kraiem

Etudiant

Med Amin Haouas

Etudiant

Rami Kaabi

Table des matieres

1. Presentation generale	3
1.1. Objectif	3
1.2. Application support	3
1.3. Stack et division des roles	3
1.4. Architecture finale	3
2. Exercice 1 — Integration Continue et Qualite du Code	5
2.1. Mise en place de Jenkins	5
2.2. Application demo et premieres briques	5
2.3. Configuration de SonarCloud	6
2.4. Stages 1 a 5 du Jenkinsfile	6
3. Exercice 2 — Livraison Continue : Artefacts et Securite	9
3.1. Dockerfile	9
3.2. Credentials Docker Hub	9
3.3. Stages 6 a 8 du Jenkinsfile	9
4. Exercice 3 — Deploiement Complet : IaC et Orchestration	12
4.1. Provisionnement avec Terraform	12
4.2. Deploiement applicatif avec Ansible	12
4.3. Stages 9 a 11 du Jenkinsfile	13
5. Exercice 4 — Observabilite : Prometheus & Grafana	16
5.1. Installation de la stack via Helm/Terraform	16
5.2. Collecte des metriques applicatives	16
5.3. Tableau de bord Grafana personnalise	17
5.4. Alerte AppDown via AlertManager et Resend	17
5.5. Verification de bout en bout	18
6. Conclusion	19

Presentation generale

Objectif

Le TP demande la mise en place d'une chaine DevOps complete autour d'une application web : automatiser la qualite du code, la creation d'un artefact conteneurise sur, le provisionnement de l'infrastructure, le deployment applicatif et la supervision. Le tout pilote par un Jenkinsfile unique.

Application support

Nous avons choisi une petite API `FastAPI` (Python 3.12, geree par `uv`) qui expose trois routes :

- `GET /` : retourne un JSON avec un message, le nom du pod et un compteur local de requetes ;
- `GET /healthz` : sonde de vivacite utilisee par Kubernetes ;
- `GET /metrics` : exposition Prometheus du compteur applicatif et des metriques par default du processus Python.

Le code applicatif tient en deux fichiers (`app/main.py` + `app/routes.py`) et 3 tests `pytest` dont la couverture est rapportee a SonarCloud.

Stack et division des roles

Orchestration	Jenkins (image personnalisee, en docker compose)
Qualite	SonarCloud (SaaS, scanner CLI execute en CI)
Artefact	Docker + Trivy + Docker Hub
Infrastructure	Terraform : kind cluster, namespace, ingress-nginx, kube-prometheus-stack
Configuration	Ansible : Deployment, Service, Ingress, ServiceMonitor, PrometheusRule, dashboard Grafana
Supervision	Prometheus + Grafana + AlertManager (livres par <code>kube-prometheus-stack</code>)
Notification	Resend (SMTP transactionnel) pour l'email d'alerte

Architecture finale

Le diagramme ci-dessous resume les conteneurs presents sur l'hote, le cluster kind embarque et les flux entre Jenkins, le cluster et l'exterieur.

```
laptop (host)
├── Docker daemon
│   └── Network: kind
│       └── devops-cicd-lab-control-plane (le node Kubernetes)
│           ├── ingress-nginx (Helm via Terraform)
│           ├── monitoring (kube-prometheus-stack)
│           └── Prometheus
```

```

├── Grafana
├── AlertManager
├── kube-state-metrics
├── node-exporter
├── demo (deploye par Ansible)
│   ├── 2 pods FastAPI
│   ├── Service + Ingress (app.127-0-0-1.nip.io)
│   ├── ServiceMonitor (scrape config)
│   └── PrometheusRule (alerte AppDown)
├── Network: devops-tp4_default + kind
│   └── jenkins (image custom : uv, sonar-scanner, kubectl,
│               helm, terraform, trivy, ansible)
└── URLs accessibles depuis le navigateur
    ├── http://localhost:8090 Jenkins
    ├── http://app.127-0-0-1.nip.io Application
    ├── http://grafana.127-0-0-1.nip.io Grafana
    ├── http://prometheus.127-0-0-1.nip.io Prometheus
    └── http://alertmanager.127-0-0-1.nip.io AlertManager

```

Choix techniques notables

- **uv** remplace pip pour la gestion des dependances Python : il fournit un lock-file reproductible et une vitesse d'install sensiblement plus elevee.
- **kind** (Kubernetes-in-Docker) plutot qu'une VM minikube : plus rapide, scriptable depuis Terraform, ferme le terrain pour brancher Jenkins sur le meme reseau Docker.
- **Resend** plutot qu'un MailHog local : evite de connecter MailHog au reseau du cluster et donne un retour reel via la boite mail.

Exercice 1 — Integration Continue et Qualite du Code

Le premier exercice met en place les premiers etages du Jenkinsfile : recuperation du code, installation, tests unitaires, analyse statique SonarCloud et Quality Gate bloquant.

Mise en place de Jenkins

Jenkins est livre via une image Docker personnalisee qui pre-installe tous les outils dont les etapes futures auront besoin (uv, sonar-scanner, docker CLI, kubectl, helm, terraform, trivy, ansible). L'image est decrite dans `ci/jenkins.Dockerfile` et instanciee par `docker-compose.ci.yml`.

```
docker compose -f docker-compose.ci.yml up -d --build
```

L'assistant Jenkins demande un mot de passe initial, l'installation des plugins suggeres puis l'ajout du plugin `SonarQube Scanner for Jenkins`.

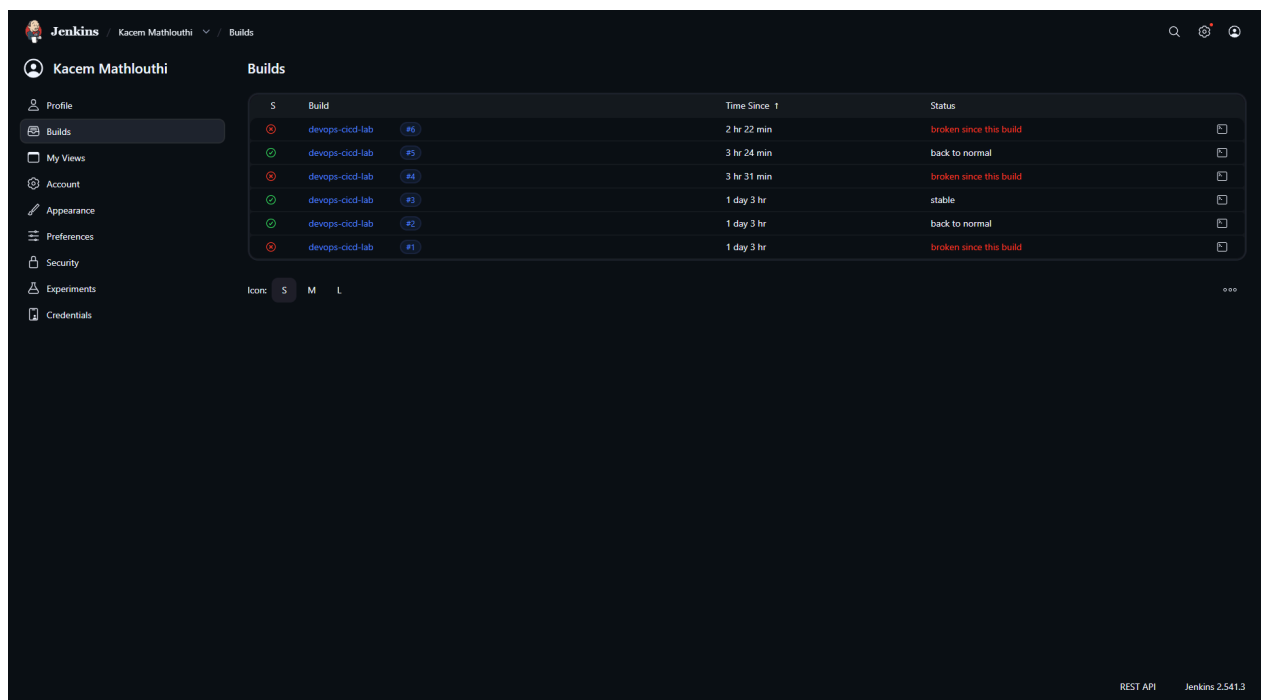


Fig. 1. – Tableau de bord Jenkins apres premier login

Application demo et premieres briques

L'application FastAPI vit dans `app/`. La gestion des dependances repose sur `uv` et un `pyproject.toml` minimaliste.

```
uv sync --frozen
uv run pytest
```

Les trois tests valident chacune des routes (`/`, `/healthz`, `/metrics`) et écrivent un `coverage.xml` qui sera lu par SonarCloud.

Configuration de SonarCloud

Après import du dépôt dans SonarCloud (organisation `kacemmathlouthi`, cle de projet `KacemMathlouthi_devops-cicd-lab`) et génération d'un token, nous déclarons dans Jenkins :

- une credential `Secret text` d'identifiant `sonar-token` ;
- un serveur SonarQube nommé exactement `SonarCloud` pointant vers `https://sonarcloud.io` et utilisant ce token.

Le Quality Gate par défaut Sonar way (couverture du nouveau code $\geq 80\%$, 0 nouvelle vulnérabilité, note A) est conservé.

Stages 1 à 5 du Jenkinsfile

Le Jenkinsfile exécute, dans l'ordre :

- 1 `Checkout` `git clone + capture du SHA court`
- 2 `Install` `uv sync -frozen`
- 3 `Unit Tests` `pytest -junitxml + coverage.xml`
- 4 `Static Analysis` `sonar-scanner` sous `withSonarQubeEnv`
- 5 `Quality Gate` `waitForQualityGate abortPipeline: true`

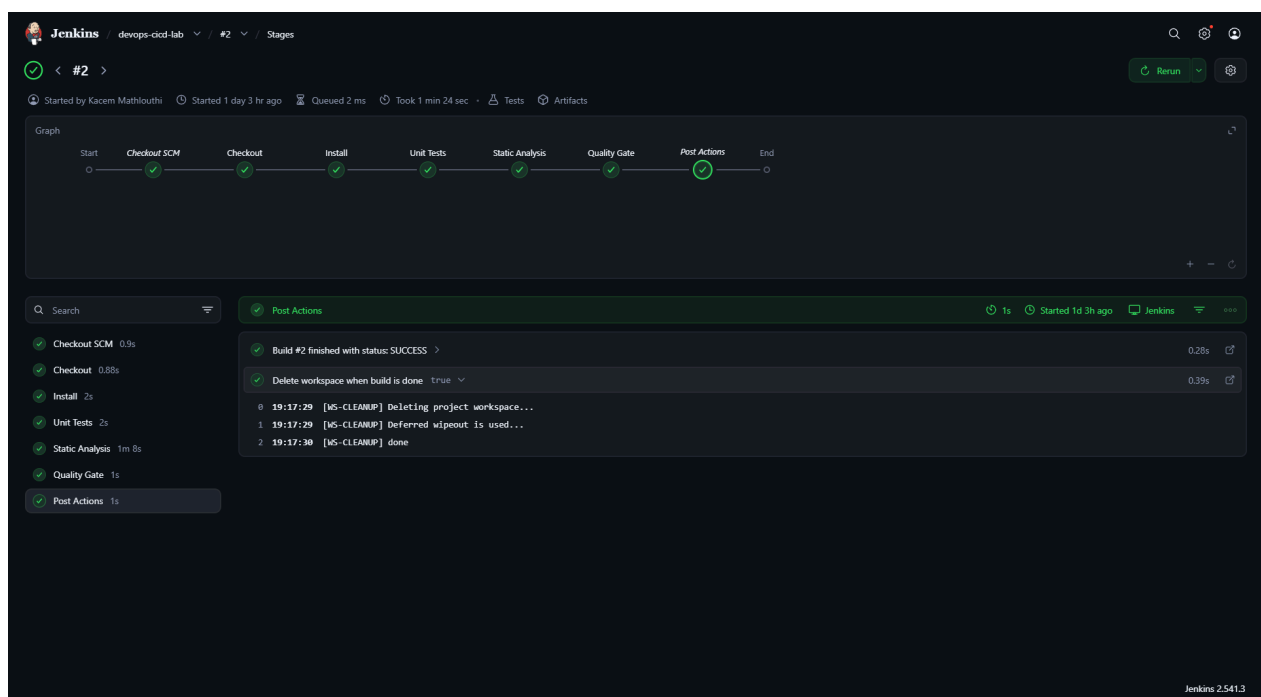
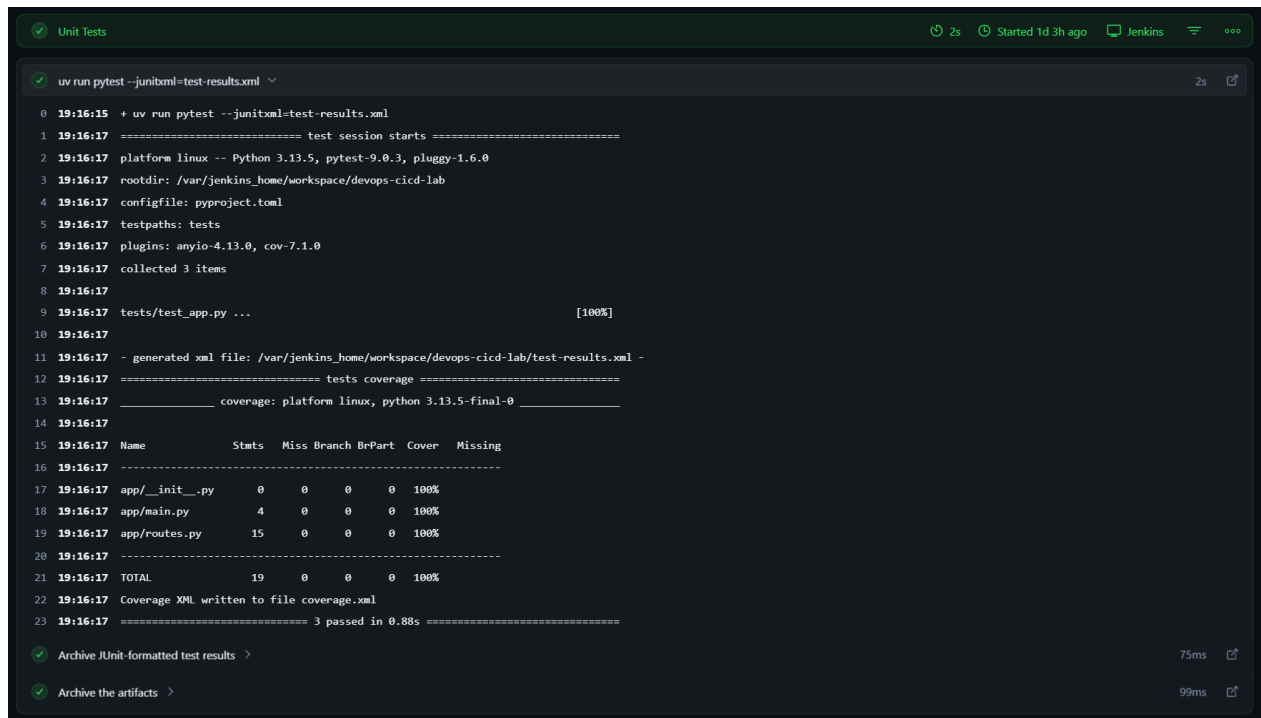


Fig. 2. – Vue Stage View de Jenkins limitée aux cinq premiers étages



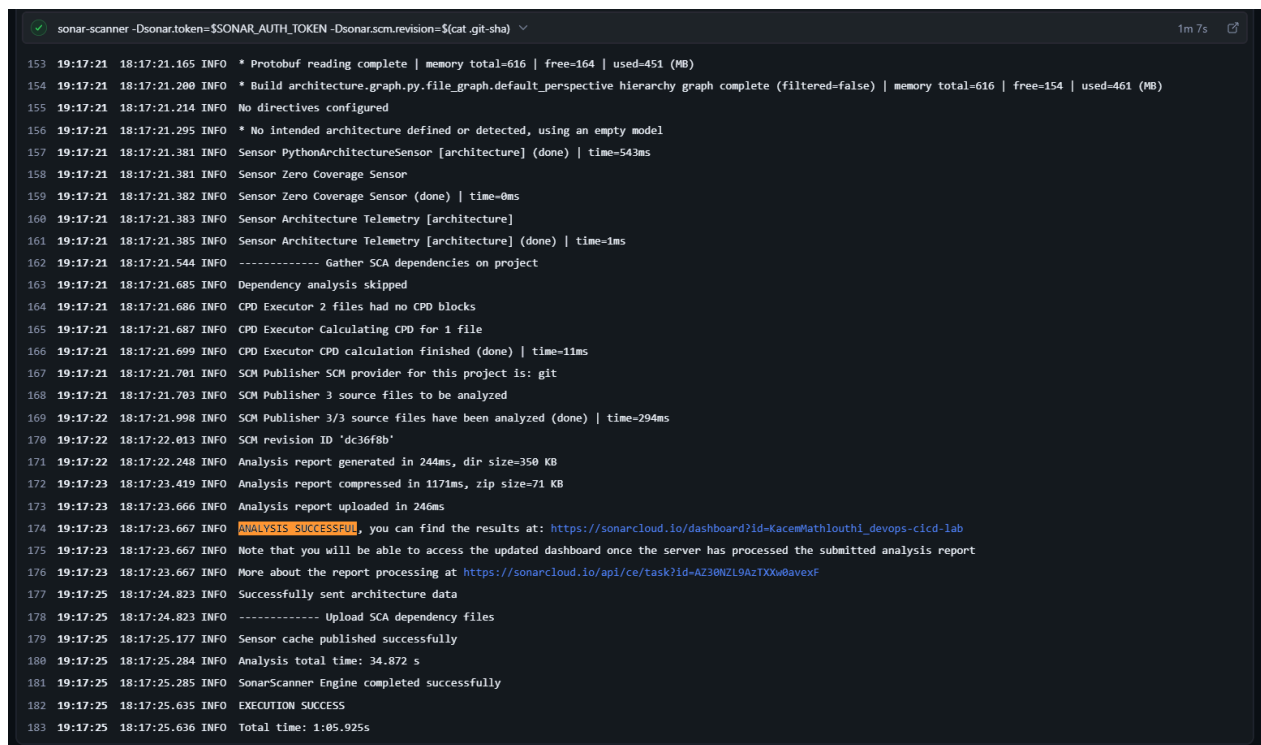
```
Unit Tests 2s Started 1d 3h ago Jenkins
```

```
uv run pytest --junitxml=test-results.xml
```

```
0 19:16:15 + uv run pytest --junitxml=test-results.xml
1 19:16:17 ===== test session starts =====
2 19:16:17 platform linux -- Python 3.13.5, pytest-9.0.3, pluggy-1.6.0
3 19:16:17 rootdir: /var/jenkins_home/workspace/devops-cicd-lab
4 19:16:17 configfile: pyproject.toml
5 19:16:17 testpaths: tests
6 19:16:17 plugins: anyio-4.13.0, cov-7.1.0
7 19:16:17 collected 3 items
8 19:16:17
9 19:16:17 tests/test_app.py ... [100%]
10 19:16:17
11 19:16:17 - generated xml file: /var/jenkins_home/workspace/devops-cicd-lab/test-results.xml -
12 19:16:17 ===== tests coverage =====
13 19:16:17 _____ coverage: platform linux, python 3.13.5-final-0 _____
14 19:16:17
15 19:16:17 Name Stmts Miss Branch BrPart Cover Missing
16 19:16:17 -----
17 19:16:17 app/__init__.py 0 0 0 0 100%
18 19:16:17 app/main.py 4 0 0 0 100%
19 19:16:17 app/routes.py 15 0 0 0 100%
20 19:16:17 -----
21 19:16:17 TOTAL 19 0 0 0 100%
22 19:16:17 Coverage XML written to file coverage.xml
23 19:16:17 ===== 3 passed in 0.88s =====
```

Archive JUnit-formatted test results 75ms

Archive the artifacts 99ms

Fig. 3. – Console Jenkins : sortie de `pytest` (3 tests passes) avec rapport de couverture

```
sonar-scanner -Dsonar.token=$SONAR_AUTH_TOKEN -Dsonar.scm.revision=${cat .git-sha} 1m 7s
```

```
153 19:17:21 18:17:21.165 INFO * Protobuf reading complete | memory total=616 | free=164 | used=451 (MB)
154 19:17:21 18:17:21.200 INFO * Build architecture.graph.py.file_graph.default_perspective_hierarchy graph complete (filtered=false) | memory total=616 | free=154 | used=461 (MB)
155 19:17:21 18:17:21.214 INFO No directives configured
156 19:17:21 18:17:21.295 INFO * No intended architecture defined or detected, using an empty model
157 19:17:21 18:17:21.381 INFO Sensor PythonArchitectureSensor [architecture] (done) | time=543ms
158 19:17:21 18:17:21.381 INFO Sensor Zero Coverage Sensor
159 19:17:21 18:17:21.382 INFO Sensor Zero Coverage Sensor (done) | time=0ms
160 19:17:21 18:17:21.383 INFO Sensor Architecture Telemetry [architecture]
161 19:17:21 18:17:21.385 INFO Sensor Architecture Telemetry [architecture] (done) | time=1ms
162 19:17:21 18:17:21.544 INFO ----- Gather SCA dependencies on project
163 19:17:21 18:17:21.685 INFO Dependency analysis skipped
164 19:17:21 18:17:21.686 INFO CPD Executor 2 files had no CPD blocks
165 19:17:21 18:17:21.687 INFO CPD Executor Calculating CPD for 1 file
166 19:17:21 18:17:21.699 INFO CPD Executor CPD calculation finished (done) | time=11ms
167 19:17:21 18:17:21.701 INFO SCM Publisher SCM provider for this project is: git
168 19:17:21 18:17:21.703 INFO SCM Publisher 3 source files to be analyzed
169 19:17:22 18:17:21.998 INFO SCM Publisher 3/3 source files have been analyzed (done) | time=294ms
170 19:17:22 18:17:22.013 INFO SCM revision ID 'dc36f8b'
171 19:17:22 18:17:22.248 INFO Analysis report generated in 244ms, dir size=350 KB
172 19:17:23 18:17:23.419 INFO Analysis report compressed in 1171ms, zip size=71 KB
173 19:17:23 18:17:23.666 INFO Analysis report uploaded in 246ms
174 19:17:23 18:17:23.667 INFO ANALYSIS SUCCESSFUL, you can find the results at: https://sonarcloud.io/dashboard?id=KacemMathlouthi_devops-cicd-lab
175 19:17:23 18:17:23.667 INFO Note that you will be able to access the updated dashboard once the server has processed the submitted analysis report
176 19:17:23 18:17:23.667 INFO More about the report processing at https://sonarcloud.io/api/ce/task?id=AZ30NZL9AZTX0w0avexF
177 19:17:25 18:17:24.823 INFO Successfully sent architecture data
178 19:17:25 18:17:24.823 INFO ----- Upload SCA dependency files
179 19:17:25 18:17:25.177 INFO Sensor cache published successfully
180 19:17:25 18:17:25.284 INFO Analysis total time: 34.872 s
181 19:17:25 18:17:25.285 INFO SonarScanner Engine completed successfully
182 19:17:25 18:17:25.635 INFO EXECUTION SUCCESS
183 19:17:25 18:17:25.636 INFO Total time: 1:05.925s
```

Fig. 4. – Console Jenkins : execution de `sonar-scanner` et message `ANALYSIS SUCCESSFUL`

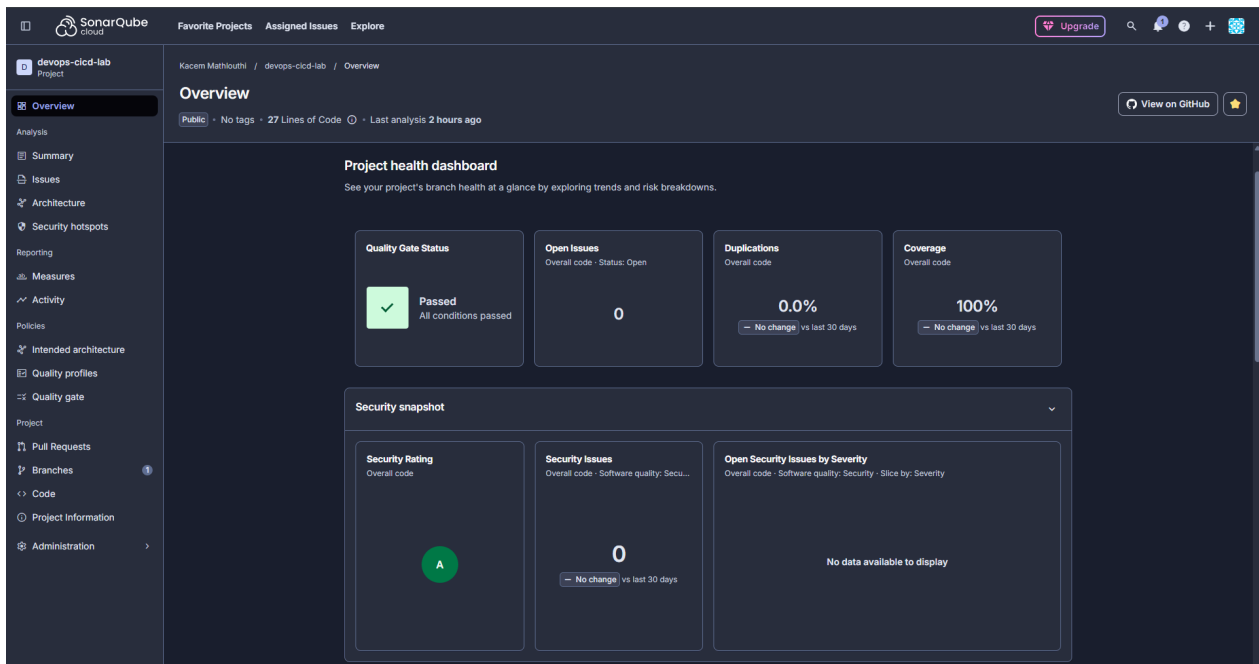


Fig. 5. – Page projet SonarCloud : Quality Gate Passed, notes A en Securite/Reliability/Maintainability, couverture 100 %

Resultat

La pipeline construit, teste, analyse et bloque sur la qualite. Un commit qui ferait baisser la couverture en dessous de 80 % ou qui introduirait un nouveau bug echouerait au stage Quality Gate, ce qui satisfait l'objectif de l'exercice 1.

Exercice 2 — Livraison Continue : Artefacts et Sécurité

L'exercice 2 ajoute trois etages au Jenkinsfile : construction d'une image Docker, scan de vulnerabilites avec Trivy puis publication sur Docker Hub.

Dockerfile

Le Dockerfile est multi-stage : un etage build base sur `ghcr.io/astral-sh/uv:python3.12-bookworm-slim` qui execute `uv sync -frozen -no-dev -no-install-project`, puis un etage runtime `python:3.12-slim` minimaliste qui copie uniquement le venv et le code applicatif. Le conteneur tourne sous un utilisateur non-root, expose le port `5000` et inclut un `HEALTHCHECK` HTTP sur `/healthz`.

Credentials Docker Hub

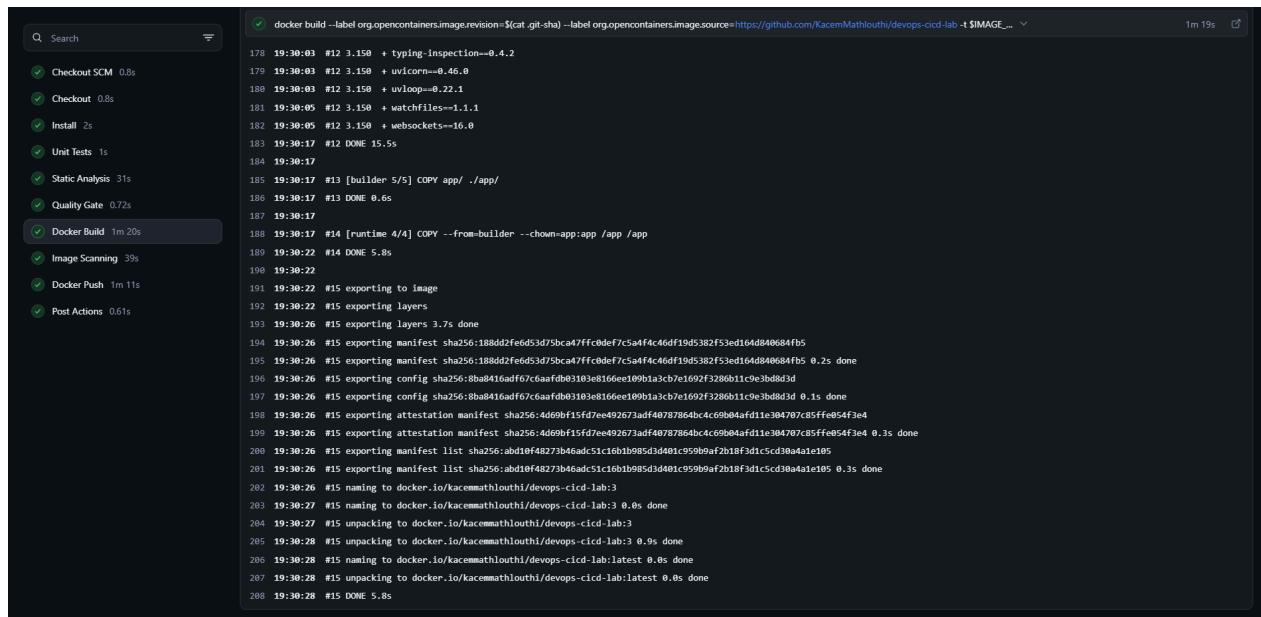
Dans `hub.docker.com`, creation d'un Personal Access Token (Read & Write). Cote Jenkins, ajout d'une credential `Username with password` d'identifiant `dockerhub-creds`.

Stages 6 a 8 du Jenkinsfile

```
6 Docker Build tags ${BUILD_NUMBER} et latest, labels OCI image.revision et
  image.source
7 Image Scanning trivy image : rapport HIGH+CRITICAL, echec uniquement sur CRITICAL corri-
  geable
8 Docker Push docker login -password-stdin, docker push des deux tags,
  docker logout
```

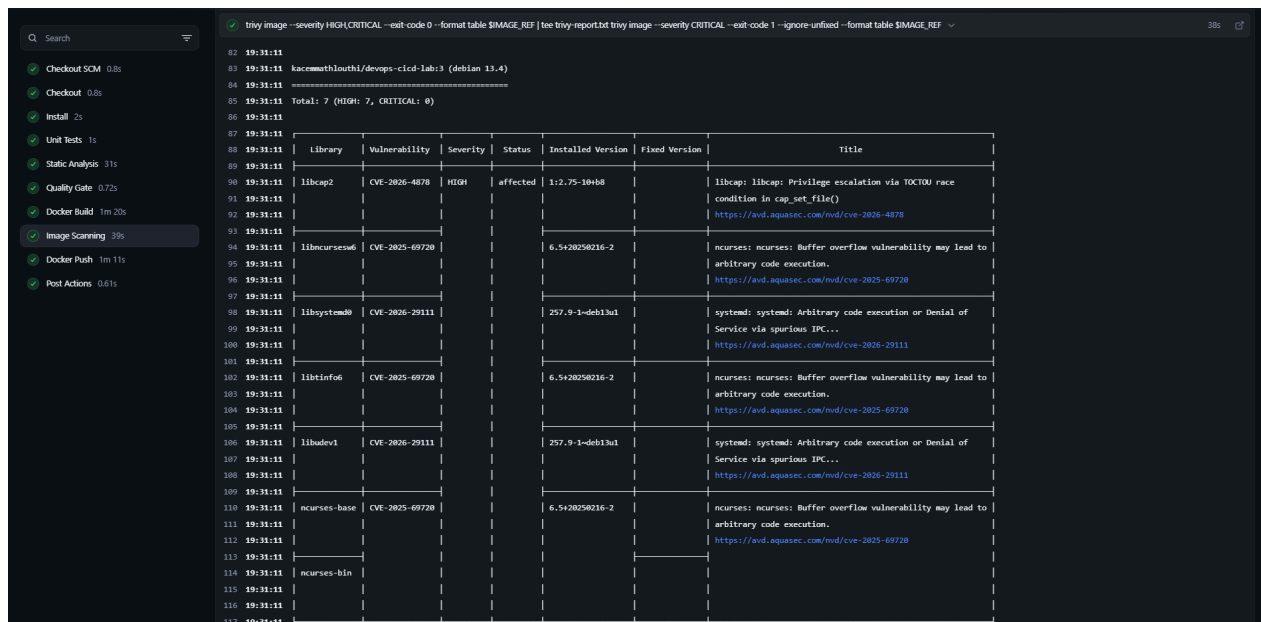
Note

Trivy est invoque deux fois par stage : un premier appel descriptif (HIGH+CRITICAL, sans echec) genere un rapport archivable; un second appel restreint aux CRITIQUES corrigeables (`-ignore-unfixed`) decide de l'echec. Cela demontre la barriere securite tout en laissant passer les CVEs intransigeantes des images de base.



```
docker build --label org.opencontainers.image.revision=${cat .git-sha} --label org.opencontainers.image.source=https://github.com/KacemMathlouthi/devops-cicd-lab -t $IMAGE_REF .
```

178 19:30:03 #12 3.150 + typing-inspection==0.4.2
179 19:30:03 #12 3.150 + uvicorn==0.46.0
180 19:30:03 #12 3.150 + uvloop==0.22.1
181 19:30:05 #12 3.150 + watchfiles==1.1.1
182 19:30:05 #12 3.150 + websockets==10.0
183 19:30:17 #12 DONE 15.5s
184 19:30:17
185 19:30:17 #13 [builder 5/5] COPY app/ ./app/
186 19:30:17 #13 DONE 0.6s
187 19:30:17
188 19:30:17 #14 [runtime 4/4] COPY --from=builder --chown=app:app /app /app
189 19:30:22 #14 DONE 5.8s
190 19:30:22
191 19:30:22 #15 exporting to image
192 19:30:22 #15 exporting layers
193 19:30:26 #15 exporting layers 3.7s done
194 19:30:26 #15 exporting manifest sha256:188dd2fed53d75bca47ffc0def7c5a44c46df19d5382f53ed164d84084fb5
195 19:30:26 #15 exporting manifest sha256:188dd2fed53d75bca47ffc0def7c5a44c46df19d5382f53ed164d84084fb5 0.2s done
196 19:30:26 #15 exporting config sha256:8ba8416adf67c6aafdb03103e8166ee109b1a3cb7e1692f3286b1c9e3bd8d3d
197 19:30:26 #15 exporting config sha256:8ba8416adf67c6aafdb03103e8166ee109b1a3cb7e1692f3286b1c9e3bd8d3d 0.1s done
198 19:30:26 #15 exporting attestation manifest sha256:4d69bf15f47ee492673ad40787864bc4c69b04afd11e384707c85ffe054f3e4
199 19:30:26 #15 exporting attestation manifest sha256:4d69bf15f47ee492673ad40787864bc4c69b04afd11e384707c85ffe054f3e4 0.3s done
200 19:30:26 #15 exporting manifest list sha256:abd10f48273b46adc51c1b1b985d3d401c959b9af2b18f3dc5cd38a4a1e105
201 19:30:26 #15 exporting manifest list sha256:abd10f48273b46adc51c1b1b985d3d401c959b9af2b18f3dc5cd38a4a1e105 0.3s done
202 19:30:27 #15 naming to docker.io/kacemmathlouthi/devops-cicd-lab:3
203 19:30:27 #15 naming to docker.io/kacemmathlouthi/devops-cicd-lab:3 0.8s done
204 19:30:27 #15 unpacking to docker.io/kacemmathlouthi/devops-cicd-lab:3
205 19:30:28 #15 unpacking to docker.io/kacemmathlouthi/devops-cicd-lab:3 0.9s done
206 19:30:28 #15 naming to docker.io/kacemmathlouthi/devops-cicd-lab:latest 0.8s done
207 19:30:28 #15 unpacking to docker.io/kacemmathlouthi/devops-cicd-lab:latest 0.8s done
208 19:30:28 #15 DONE 5.8s

Fig. 6. – Console Jenkins : sortie de `docker build` (multi-stage) avec les deux tags

```
trivy image --severity HIGH/CRITICAL --exit code 0 --format table $IMAGE_REF | tee trivy-report.txt trivy image --severity CRITICAL --exit code 1 --ignore-unfixed --format table $IMAGE_REF
```

Library	Vulnerability	Severity	Status	Installed Version	Fixed Version	Title
libcap2	CVE-2026-4078	HIGH	affected	1:2.75-10+8		libcap: libcap: Privilege escalation via TOCTOU race condition in cap_set_file() https://avd.aquasec.com/nvd/cve-2026-4078
libcurses6	CVE-2025-69720			6.5+20250216-2		ncurses: ncurses: Buffer overflow vulnerability may lead to arbitrary code execution. https://avd.aquasec.com/nvd/cve-2025-69720
libsystemd0	CVE-2026-29111			257.9-1-deb1h1		systemd: systemd: Arbitrary code execution or Denial of Service via spurious IPC... https://avd.aquasec.com/nvd/cve-2026-29111
libtinfo6	CVE-2025-69720			6.5+20250216-2		ncurses: ncurses: Buffer overflow vulnerability may lead to arbitrary code execution. https://avd.aquasec.com/nvd/cve-2025-69720
libudev1	CVE-2026-29111			257.9-1-deb1h1		systemd: systemd: Arbitrary code execution or Denial of Service via spurious IPC... https://avd.aquasec.com/nvd/cve-2026-29111
ncurses-base	CVE-2025-69720			6.5+20250216-2		ncurses: ncurses: Buffer overflow vulnerability may lead to arbitrary code execution. https://avd.aquasec.com/nvd/cve-2025-69720
ncurses-bin						

Fig. 7. – Console Jenkins : rapport Trivy table affichant les CVEs HIGH/CRITICAL detectees

Fig. 8. – Console Jenkins : `docker push` réussi pour les deux tags vers Docker Hub

Fig. 9. – Page Docker Hub du repository `kacemmathlouthi/devops-cicd-lab` avec les tags `latest` et `${BUILD_NUMBER}`

Resultat

Chaque build genere un artefact traceable : tag immuable lie au numero de build, image scannee par Trivy, publiee publiquement sur Docker Hub. Ce sont les pieces necessaires pour que la phase de deploiement de l'exercice 3 puisse en reference une version precise.

Exercice 3 — Déploiement Complet : IaC et Orchestration

L'exercice 3 amène l'image jusqu'à un cluster Kubernetes réel (kind, en local) via Terraform pour l'infrastructure puis Ansible pour les objets applicatifs. Un smoke test final valide que l'URL publique répond.

Provisionnement avec Terraform

Le module `infra/terraform/` déclare :

- une ressource `kind_cluster` (provider `tehcyrx/kind`) qui crée un cluster à un seul nœud, étiqueté `ingress-ready=true`, avec mappage des ports 80/443 vers l'hôte ;
- un namespace `demo` (provider `hashicorp/kubernetes`) ;
- une release Helm `ingress-nginx` (provider `hashicorp/helm`) configurée pour utiliser `hostPort` et tolérer le taint control-plane ;
- une release Helm `kube-prometheus-stack` qui sera utilisée par l'exercice 4.

```
cd infra/terraform
terraform init
terraform apply
```

```
(devops-cicd-lab) kacem@Kacem:~/projects/devops-tp4/infra/terraform$ terraform apply
kind_cluster.this: Refreshing state... [id=devops-cicd-lab-]
kubernetes_namespace.demo: Refreshing state... [id=demo]
helm_release.ingress_nginx: Refreshing state... [id=ingress-nginx]
helm_release.kube_prometheus_stack: Refreshing state... [id=kps]

No changes. Your infrastructure matches the configuration.

Terraform has compared your real infrastructure against your configuration and found no differences, so no changes are needed.

Apply complete! Resources: 0 added, 0 changed, 0 destroyed.

Outputs:

alertmanager_url = "http://alertmanager.127-0-0-1.nip.io"
cluster_name     = "devops-cicd-lab"
grafana_url      = "http://grafana.127-0-0-1.nip.io"
namespace       = "demo"
prometheus_url   = "http://prometheus.127-0-0-1.nip.io"
(devops-cicd-lab) kacem@Kacem:~/projects/devops-tp4/infra/terraform$
```

Fig. 10. – Sortie `terraform apply` : ressources créées + outputs (`cluster_name`, `namespace`, URLs)

Déploiement applicatif avec Ansible

Le playbook `deploy/ansible/site.yml` applique, via `kubernetes.core.k8s`, les manifests suivants templatisés en Jinja2 :

- **Deployment** : 2 replicas, sondes `readiness` et `liveness` sur `/healthz`, annotations Prometheus, tag d'image lu depuis la variable d'environnement `IMAGE_TAG` ;
- **Service** : ClusterIP pointant vers le port `http` des pods ;
- **Ingress** : classe `nginx`, hôte `app.127-0-0-1.nip.io` .

```
set -a && source .env && set +a
ansible-playbook deploy/ansible/site.yml -e image_tag=latest
```

Stages 9 a 11 du Jenkinsfile

9	Infrastructure (Terraform)	<code>init</code> , <code>fmt -check</code> , <code>validate</code> , <code>plan -out</code> , <code>apply</code> (idempotent) sur l'état partagé avec l'hôte
10	Configure & Deploy (Ansible)	<code>ansible-playbook</code> avec <code>KUBECONFIG</code> charge depuis la credential <code>kind-kubeconfig</code> et <code>IMAGE_TAG=\${BUILD_NUMBER}</code>
11	Smoke Test	<code>curl -connect-to ... http://app.127-0-0-1.nip.io/</code> , 30 essais avec backoff

Detail technique : reseaux et etat Terraform

Jenkins tourne dans un conteneur sur le réseau Docker `devops-tp4_default` . Pour atteindre le cluster kind, on l'ajoute aussi au réseau `kind` (`networks: [default, kind]` dans le compose). Le smoke test utilise `curl -connect-to` pour conserver l'URL publique tout en redirigeant la connexion TCP vers le hostname interne du node `devops-cicd-lab-control-plane` .

Pour permettre à Jenkins d'exécuter un vrai `terraform apply` , le répertoire `infra/terraform/` est bind-monté dans le conteneur Jenkins via `/host-tf-state` . Le pipeline copie le `terraform.tfstate` dans le workspace, applique, puis le recopie : l'état est partagé entre l'hôte et Jenkins.

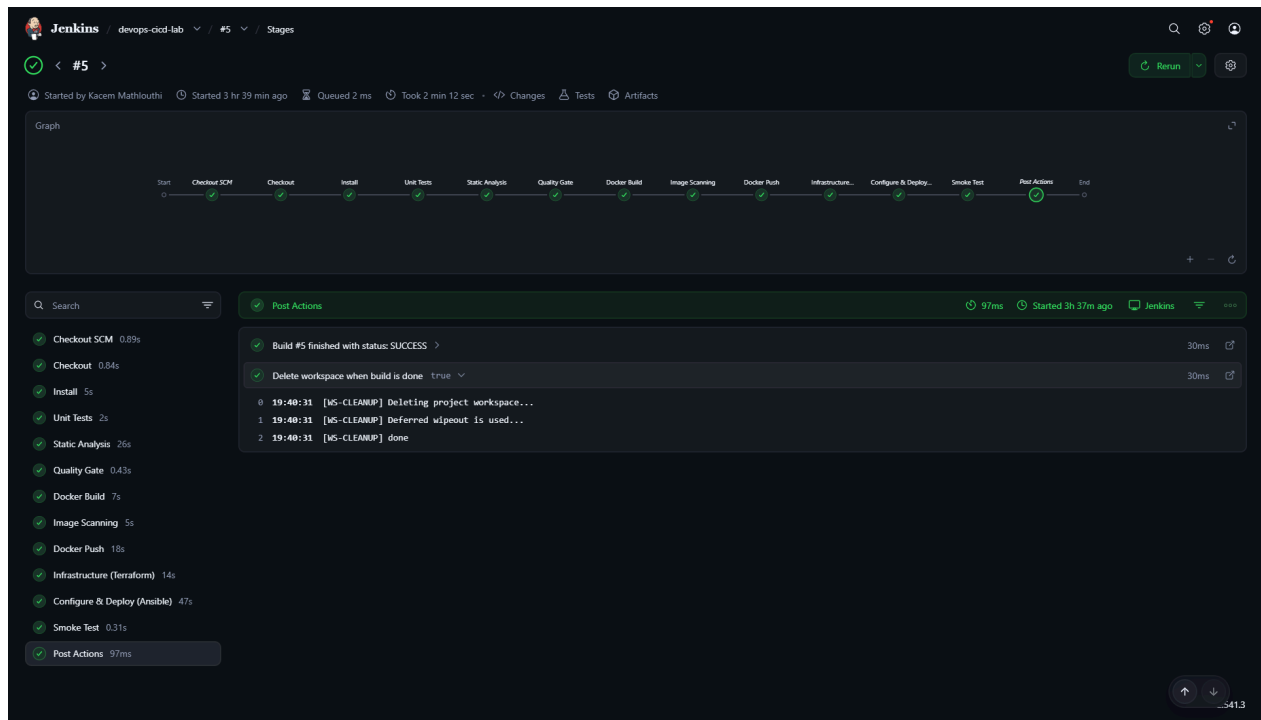


Fig. 11. – Vue Stage View Jenkins : les 11 stages au vert sur le dernier build

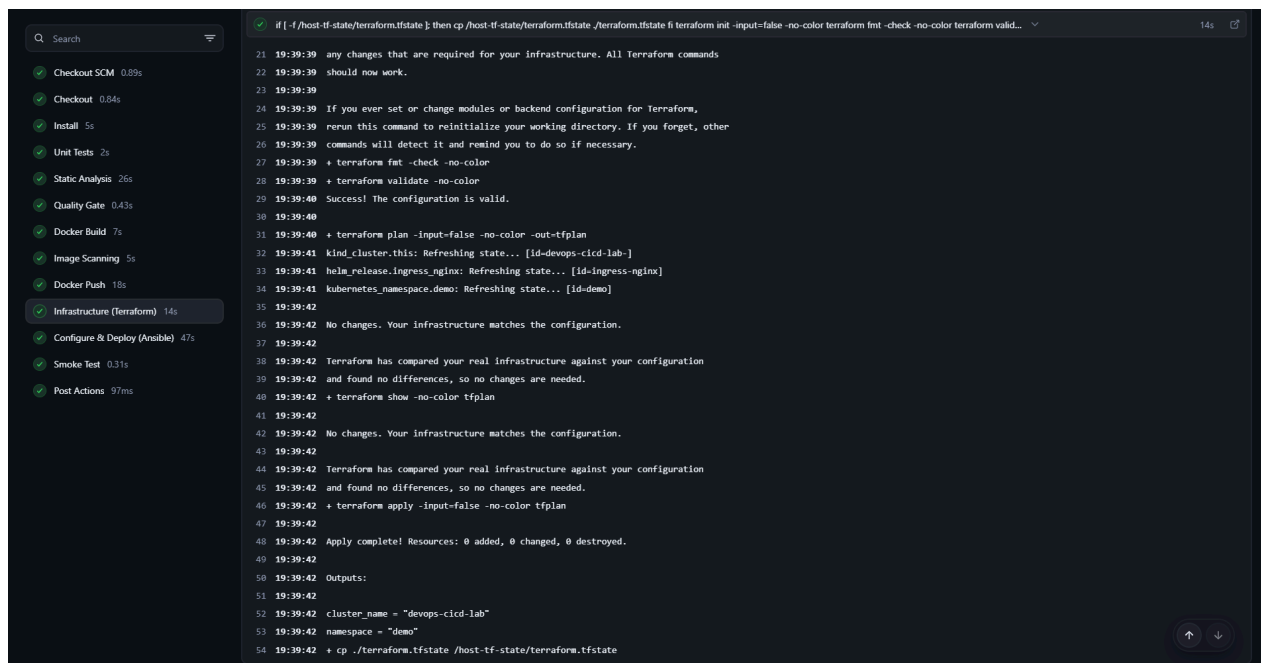


Fig. 12. – Console Jenkins : sortie du stage Terraform (plan + apply, 0 added 0 changed 0 destroyed apres premiere execution)

```

export KUBECONFIG="$KUBECONFIG_FILE" export IMAGE_TAG="$BUILD_NUMBER" ansible-playbook site.yml

0 19:39:43 + export KUBECONFIG=****
1 19:39:43 + export IMAGE_TAG=5
2 19:39:43 + ansible-playbook site.yml
3 19:39:44
4 19:39:44 PLAY [Deploy devops-cicd-lab to Kubernetes] *****
5 19:39:44
6 19:39:44 TASK [Apply Deployment] *****
7 19:40:30 changed: [localhost]
8 19:40:30
9 19:40:30 TASK [Apply Service] *****
10 19:40:30 ok: [localhost]
11 19:40:30
12 19:40:30 TASK [Apply Ingress] *****
13 19:40:30 ok: [localhost]
14 19:40:30
15 19:40:30 TASK [Show what was deployed] *****
16 19:40:30 ok: [localhost]
17 19:40:30
18 19:40:30 TASK [Print pod summary] *****
19 19:40:30 ok: [localhost] => (item=devops-cicd-lab-8547cb846-dn0kq) => {
20 19:40:30   "msg": "devops-cicd-lab-8547cb846-dn0kq -> Running"
21 19:40:30 }
22 19:40:30 ok: [localhost] => (item=devops-cicd-lab-8547cb846-mcc5f) => {
23 19:40:30   "msg": "devops-cicd-lab-8547cb846-mcc5f -> Running"
24 19:40:30 }
25 19:40:30
26 19:40:30 PLAY RECAP *****
27 19:40:30 localhost          : ok=5  changed=1  unreachable=0  failed=0  skipped=0  rescued=0  ignored=0
28 19:40:30

```

Fig. 13. – Console Jenkins : recap Ansible (PLAY RECAP : ok=N, changed=1 sur le Deployment, failed=0)

```

set -e for i in $(seq 1 30); do code=$(curl -s -o /tmp/smoke-body -w %[http_code] -H "Host: app.127-0-0-1.nip.io" http://devops-cicd-lab-control-plane/ || true) if [ "$code" = "200..." ]; then
0 19:40:31 + set -e
1 19:40:31 + seq 1 30
2 19:40:31 + curl -s -o /tmp/smoke-body -w %[http_code] -H Host: app.127-0-0-1.nip.io http://devops-cicd-lab-control-plane/
3 19:40:31 + code=200
4 19:40:31 + [ 200 - 200 ]
5 19:40:31 + echo Smoke test OK (HTTP 200)
6 19:40:31 Smoke test OK (HTTP 200)
7 19:40:31 + cat /tmp/smoke-body
8 19:40:31 {"message":"hello from devops-cicd-lab","hostname":"devops-cicd-lab-8547cb846-mcc5f","count":1}+ echo
9 19:40:31
10 19:40:31 + exit 0

```

Fig. 14. – Console Jenkins : smoke test réussi (Smoke test OK (HTTP 200) + JSON renvoie par l'application)

Resultat

La chaine est complete : un commit pousse sur la branche `main` declenche (sur Build Now) la construction de l'image, sa publication, le rapprochement de l'infrastructure, le redeploiement applicatif et la verification HTTP de l'URL exposee.

Exercice 4 — Observabilite : Prometheus & Grafana

L'exercice 4 ajoute la pile de supervision et une regle d'alerte. Tout est livré au cluster une fois pour toutes par Terraform (chart Helm) et Ansible (objets propres à l'application).

Installation de la stack via Helm/Terraform

Le fichier `infra/terraform/monitoring.tf` declare une release Helm `kube-prometheus-stack` dans le namespace `monitoring`. Cette chart agrege Prometheus, Grafana, AlertManager, kube-state-metrics, node-exporter et le Prometheus Operator.

Le service Grafana est expose via une `Ingress` sur `grafana.127-0-0-1.nip.io`, idem pour Prometheus et AlertManager.

```
cd infra/terraform
terraform apply      # ressource helm_release.kube_prometheus_stack
```

```
(devops-cicd-lab) kacem@Kacem:~/projects/devops-tp4/infra/terraform$ kubectl get pods -n monitoring
NAME                                READY   STATUS    RESTARTS   AGE
alertmanager-kps-kube-prometheus-stack-alertmanager-0  2/2     Running   0          122m
kps-grafana-7764d54967-pt5cb         3/3     Running   0          3h25m
kps-kube-prometheus-stack-operator-b888466f6-5wq8z     1/1     Running   0          3h25m
kps-kube-state-metrics-6d6bc99f97-71qb8               1/1     Running   2 (3h23m ago)  3h25m
kps-prometheus-node-exporter-vvcqs                 1/1     Running   0          3h25m
prometheus-kps-kube-prometheus-stack-prometheus-0     2/2     Running   0          3h25m
(devops-cicd-lab) kacem@Kacem:~/projects/devops-tp4/infra/terraform$
```

Fig. 15. – `kubectl get pods -n monitoring` : 6 pods Running (Prometheus, Grafana, AlertManager, Operator, kube-state-metrics, node-exporter)

Collecte des metriques applicatives

Pour que Prometheus scrape l'application, Ansible ajoute un objet `ServiceMonitor` dans le namespace `demo` avec le label `release: kps` (motif scrute par le Prometheus de la stack). Le `ServiceMonitor` pointe sur le port nomme `http` du Service applicatif et recupere le chemin `/metrics` toutes les 15 secondes.

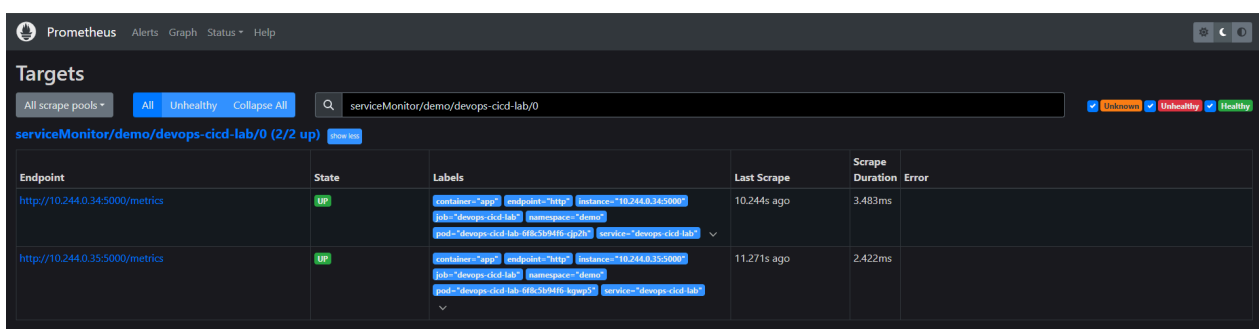


Fig. 16. – Page `Status > Targets` de Prometheus : `serviceMonitor/demo/devops-cicd-lab/0` (2/2 up), endpoints des deux pods avec etat UP

Tableau de bord Grafana personnalis 

Le dashboard est  crit en JSON et livre via un `ConfigMap` portant le label `grafana_dashboard: « 1 »`.
Le sidecar Grafana de la stack `kube-prometheus-stack` le d tecte automatiquement et l'importe.

Le tableau pr sente neuf panneaux organis s en quatre lignes :

- ligne 1 (statistiques) : Pods Up, Pod Restarts, Available/Desired Replicas, Total Requests/sec ;
- ligne 2 (ressources) : Pod CPU, Pod Memory ;
- ligne 3 (reseau) : Network Inbound, Network Outbound ;
- ligne 4 : App Request Rate by Endpoint.

Toutes les requ tes PromQL utilisent les selecteurs `namespace="demo"` et `pod=~"devops-cicd-lab-.*"` pour cibler exclusivement notre application.

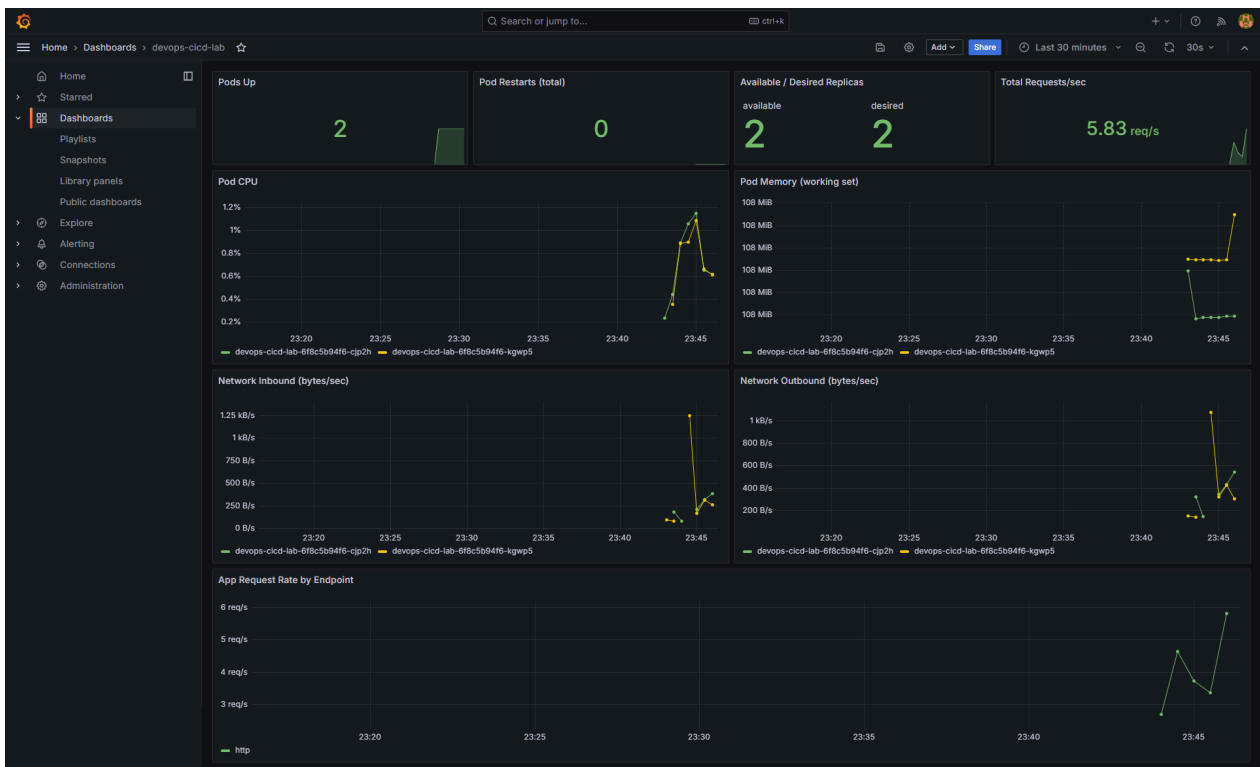


Fig. 17. – Dashboard Grafana devops-cicd-lab : neuf panneaux avec donn es vivantes (apr s une vague de `curl` sur l'application)

Alerte AppDown via AlertManager et Resend

La r gle d'alerte est pos e par Ansible (`PrometheusRule`) dans le namespace `demo` :

```
- alert: AppDown
  expr: 'absent(up{job="devops-cicd-lab",namespace="demo"}) = 1
        or sum(up{job="devops-cicd-lab",namespace="demo"}) = 0'
  for: 2m
  labels:
    severity: critical
    app: devops-cicd-lab
  annotations:
    summary: "devops-cicd-lab is DOWN"
    description: "No instances of devops-cicd-lab are reporting up=1 for more than 2 minutes."
```

AlertManager est reconfigure pour utiliser un `configSecret` genere par Ansible (`alertmanager-config`) qui contient la route et les receivers. Le SMTP cible Resend (`smtp.resend.com:587`) avec un mot de passe lu depuis un fichier (option `smtp_auth_password_file`) lui-meme issu d'un Secret Kubernetes `alertmanager-resend` monte en volume sur le pod AlertManager.

La route filtre strictement : la receiver email ne recoit que les alertes ayant `alertname = « AppDown »` ; tout le reste (alertes par default de la chart, bruit kube-system propre a kind) tombe dans le receiver `null` et n'envoie pas d'email — utile pour ne pas saturer le quota gratuit de Resend.

Verification de bout en bout

Pour valider la chaine complete, on coupe le deploiement, on attend que l'alerte se declenche (la regle a `for: 2m`), puis on retablit. AlertManager envoie un email de declenchement via Resend, suivi d'un email de resolution une fois les pods de retour grace a `send_resolved: true` .

```
kubectl --context kind-devops-cicd-lab scale deployment devops-cicd-lab -n demo --replicas=0
# Attente 2 a 3 minutes : Prometheus passe en Pending puis Firing,
# AlertManager achemine l'alerte vers Resend.
kubectl --context kind-devops-cicd-lab scale deployment devops-cicd-lab -n demo --replicas=2
```

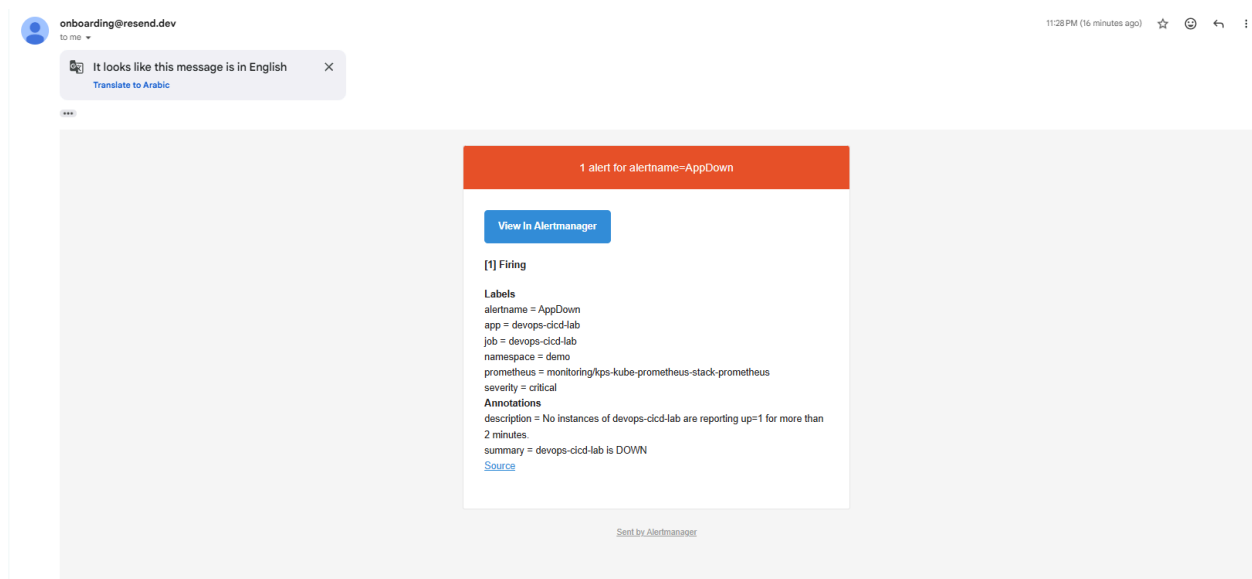


Fig. 18. – Boite mail : email `[RESOLVED]` AppDown apres remise en service, preuve que la chaine Prometheus → AlertManager → Resend a delivre l'email de declenchement puis sa resolution

Resultat

Le service est observable de bout en bout : metriques cluster et applicatives stockees dans Prometheus, dashboard Grafana personnalise, regle d'alerte qui declenche apres 2 minutes d'indisponibilite, notification email reelle puis notification de resolution. Tous les artefacts sont versionnes dans le depot et reproductibles via Terraform et Ansible.

Conclusion

Le pipeline final passe par onze stages dans un seul Jenkinsfile et touche a chaque fois aux sept outils du sujet :

Jenkins	pilote l'ensemble du pipeline en un fichier
SonarCloud	analyse statique + Quality Gate bloquant
Docker / Trivy	image multi-stage non-root, scan CRITICAL bloquant, push tagge
Terraform	cluster kind, namespace, ingress-nginx, kube-prometheus-stack
Ansible / K8s	Deployment, Service, Ingress, ServiceMonitor, PrometheusRule, dashboard
Prometheus	scrape applicatif (<code>/metrics</code>) et cluster, regle d'alerte AppDown
Grafana	dashboard auto-importe avec 9 panneaux dedies a l'application
AlertManager	route filteree + email transactionnel via Resend

Tous les livrables demandes sont presents dans le depot :

- code applicatif, Dockerfile, scripts `.tf` et playbooks Ansible ;
- `Jenkinsfile` unique contenant les onze stages ;
- ce rapport et ses captures.

Depot et URLs publiques

- GitHub : github.com/KacemMathlouthi/devops-cicd-lab
- Docker Hub : hub.docker.com/r/kacemmathlouthi/devops-cicd-lab
- SonarCloud : sonarcloud.io/project/overview?id=KacemMathlouthi_devops-cicd-lab